

CSCE 735 PARALLEL COMPUTING · MODULE 10 · LECTURE NOTES

Distributed Memory and MPI

When the data does not fit on one machine.

MODULE OUTCOMES

Explain the message-passing model and when it is necessary.

Use point-to-point and collective operations correctly.

Model communication cost and reason about it.

Overlap communication with computation.

Decompose a domain with halo exchange.

1 The message-passing model

NOTHING IS SHARED, WHICH IS A FEATURE

Each process has its own private address space. There are no data races, no memory model subtleties, and no false sharing, because there is no shared data at all — Modules 03 through 05 simply do not apply between processes.

All communication is explicit. Every byte that moves between processes appears as a call in the source code, which makes the communication cost *visible* rather than hidden inside a cache miss or a coherence transaction.

That explicitness is a substantial part of why MPI programs frequently scale better than shared-memory ones — you cannot accidentally communicate, so the decomposition has to be thought through before the program runs at all.

And it is a reasonable argument for using MPI on a single machine too. The discipline of explicit data movement tends to produce better decompositions than shared memory encourages, where it is easy to share by default and discover the cost later.

Reason given	Assessment
The data does not fit in one machine's memory.	The legitimate reason, and not negotiable. If the working set is ten terabytes, no amount of single-node tuning helps.
More cores are needed than one node provides.	Legitimate past a point — though a modern node has 128 cores and the Amdahl analysis of Module 01 should be done first.

Reason given	Assessment
Fault tolerance.	Partly, and be careful — standard MPI is <i>not</i> fault tolerant. A single process failure aborts the job, and checkpointing is the usual answer rather than anything MPI provides.
Throughput across many independent jobs.	Use a job scheduler, not MPI. Embarrassingly parallel work needs no communication, and writing it in MPI adds complexity for nothing. This is a common mistake.
It seems more serious.	No.

A single node with two terabytes of memory and 128 cores handles a great deal, and CSCE 608's Module 13 made exactly this argument about distributed databases. The reflex to distribute is costly and frequently unexamined.

2 Communication

```
// BLOCKING -- simple, and can deadlock if every rank sends first
MPI_Send(buf, n, MPI_DOUBLE, dest, tag, comm);
MPI_Recv(buf, n, MPI_DOUBLE, src, tag, comm, &status);

// NON-BLOCKING -- post, compute, wait. This is how you overlap.
MPI_Irecv(ghost, n, MPI_DOUBLE, left, 0, comm, &req[0]);
MPI_Isend(edge, n, MPI_DOUBLE, right, 0, comm, &req[1]);
compute_interior();           // overlapped with the transfer
MPI_Waitall(2, req, MPI_STATUSES_IGNORE);
compute_boundary();           // needs the ghost data

// COLLECTIVES -- use these
MPI_Allreduce(&local, &global, 1, MPI_DOUBLE, MPI_SUM, comm);
```

The blocking deadlock is worth understanding. If every rank calls `MPI_Send` before any calls `MPI_Recv`, and the messages are large enough that the implementation cannot buffer them, every rank blocks waiting for a matching receive that nobody has posted. **It works for small messages and deadlocks for large ones**, which makes it a particularly unpleasant bug to encounter in production.

USE COLLECTIVES RATHER THAN WRITING YOUR OWN

A reduction written as a loop of point-to-point sends to rank 0 is $O(p)$. MPI_Reduce is $O(\log p)$, using a tree.

And the implementation knows the network topology. It selects between different algorithms at runtime depending on message size, process count, and whether the ranks are on the same node, the same switch, or across racks — decisions you cannot easily replicate and would have to revisit on every new machine.

Some interconnects offload reductions into the network hardware, performing the combination in the switches as the data passes through. No hand-written implementation can reach that.

So learn the set: Bcast, Reduce, Allreduce, Scatter, Gather, Allgather, Alltoall, Barrier, and the scan variants. **They are almost always what you want,** and recognising which collective a piece of communication is turns out to be most of the design work.

$$T = \alpha + n\beta \quad n^* = \alpha/\beta$$

where α is the per-message latency and β the per-byte cost. Typical modern figures are α of 1 to 10 microseconds and bandwidth of 10 to 200 GB/s. **The crossover message size n^* — where latency and bandwidth contribute equally — is usually a few kilobytes.** Below it you are latency-bound and aggregating messages pays; above it you are bandwidth-bound and reducing bytes pays. **Send fewer, larger messages,** because α is charged per message regardless of size.

3 Domain decomposition and halo exchange

THE SURFACE-TO-VOLUME ARGUMENT

Each process owns a subdomain of the problem and needs a layer of its neighbours' boundary values to compute its own boundary — the **halo** or ghost cells. Each timestep, the halos are exchanged.

Computation scales with the volume of the subdomain; communication scales with its surface. This single observation governs the scaling behaviour of essentially every grid-based parallel application.

So the communication-to-computation ratio improves as the subdomain grows — which is exactly why weak scaling works well (subdomains stay the same size) and strong scaling degrades (subdomains shrink, so surface comes to dominate volume). **Module 01's two laws, explained by geometry.**

And it is why the decomposition's shape matters. Slicing a cube into flat slabs gives each process two large faces; cutting it into small cubes gives six small ones, with far less total area.

Decomposition	Surface area per process	Behaviour at large p
1D slabs	$2n^2$ — two faces, each the full cross section.	Constant in p. The communication volume per process does not fall at all as processes are added, so it rapidly dominates.
2D columns	$4n^2/\sqrt{p}$	Falls as $p^{-1/2}$. Considerably better.
3D blocks	$6n^2/p^{2/3}$	Falls as $p^{-2/3}$ — best by a wide margin. At a thousand processes on an n^3 grid this moves roughly an order of magnitude less data than 1D.

This is why every production grid code uses 3D decomposition despite the additional bookkeeping — twenty-six neighbours rather than two, and corner and edge halos as well as faces. The asymptotic difference is decisive at scale.

OVERLAP, AND THEN MEASURE WHETHER IT ACTUALLY HAPPENED

The standard idiom: post the non-blocking receives and sends, compute the *interior* of the subdomain (which needs no halo data), wait for the transfers, then compute the *boundary*. The interior computation covers the communication latency.

But posting a non-blocking operation does not guarantee that progress is being made on it. Many MPI implementations only advance transfers while inside an MPI call — there is no background thread — so a long stretch of pure computation may see no progress at all, and the entire transfer happens inside the Waitall.

So the overlap may simply not occur, and the code is correct and well-structured either way. Nothing in the source distinguishes the two cases.

Measure it. Time the interior computation with outstanding communication and without. **If the two are the same, nothing overlapped** — and the remedies are implementation-specific: enabling asynchronous progress threads, calling MPI_Test periodically during the computation, or accepting it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Eijkhout — Parallel Programming in MPI and OpenMP (free book)	Free, thorough, and the best single reference for this module. The exercises are good.
MPI Standard and the MPICH / OpenMPI documentation (free)	The specification, plus two implementations' guides. The collectives chapter is worth reading in full.
Hoefler et al. — performance modelling and MPI work (free)	The cost model of §2 and measurement methodology, from a group that is unusually rigorous about both.
Grama, Gupta, Karypis & Kumar — Introduction to Parallel Computing	The decomposition analysis of §3, with the surface-to-volume derivations done properly.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write an MPI hello-world, then a ring communication, and confirm both run across multiple processes.
- 2 **Construct the blocking deadlock:** have every rank send before receiving, and increase the message size until it hangs. Report the threshold.
- 3 Implement a reduction with point-to-point messages and compare against MPI_Reduce at 4, 16, and 64 processes.
- 4 Measure α and β for your interconnect by timing messages of many sizes. Fit the model and compute n^* .
- 5 Implement 1D and 3D domain decomposition for a 3D stencil. Measure the communication volume per process for each at several process counts.
- 6 Implement halo exchange with blocking and with non-blocking communication, and compare.
- 7 **Measure whether your overlap actually occurred** by timing the interior computation with and without outstanding communication.
- 8 Run a strong scaling study and a weak scaling study on the same stencil code and explain the difference using the surface-to-volume argument.
- 9 Take a problem you solved with threads in Project 1 and estimate honestly whether MPI would help.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does the message-passing model provide, and what does its explicitness buy?
- 2 Give five reasons people distribute and assess each.
- 3 How does a blocking send deadlock, and why is it size-dependent?
- 4 Give three reasons to use collectives rather than hand-written communication.
- 5 State the communication cost model and the crossover message size.
- 6 Explain the surface-to-volume argument and what it says about weak and strong scaling.
- 7 Compare 1D, 2D, and 3D decomposition on communication volume.
- 8 Describe the overlap idiom and say why it may not work.